The Chaos Language Algorithm: Exact-Reconstructing Grammar Induction for Symbolic Dynamics

Chaos Language Algorithm Project

Evidence cutoff: 2026-07-12

Abstract

The Chaos Language Algorithm (CLA) is a program for converting a continuous or discrete trajectory into a symbolic sequence and inducing a compact, exactly reconstructing grammar. Its central distinction is between syntagmatic compression by chunks, which abbreviate repeated sequences, and paradigmatic compression by categories, which group contextually substitutable tokens while retaining the selected member at every occurrence. This paper gives an evidence-bounded account of the implementation through repository commit `16fe6bc`. The pure-Python `chaoslang` prototype implements typed grammar state, bounded-suffix-trie chunk proposals, exact and Jensen–Shannon context categories, a greedy two-part MDL-like search, exact expansion checks, persistence and replay, a backend-neutral fact projection, fixed-grid and adaptive symbolization, Phase-0 surrogate and held-out diagnostics, a dependency-free TICA/VAMP-style kinetic map, and an optional `deeptime` backend. Recorded fixed-grid controls cover five dynamical systems and Lorenz-96 dimensions 5–24; all eleven recorded project/repository runs report exact reconstruction. A 1024-step, 20-dimensional Lorenz-96 smoke run observed 510 symbols, eight chunk rules, score 1021.2, and no accepted category edit. The current suite was independently rerun at `16fe6bc`: 102 tests passed in 38.181 seconds. High-dimensional embedding components are implemented and unit-tested, but the planned 256-dimensional ground-truth validation and rate-distortion sweeps have not been recorded. We therefore distinguish throughout among Observed records, Reproduced checks, and Proposed research.

1 Scope and epistemic labels

This paper consolidates the implementation-oriented CLA specification, the Hyperon-ready architecture, the high-dimensional embedding design, project records, repository code and history, tests, and experiment records. The source specification is a local working document rather than a peer-reviewed publication. We use three labels.

- **Observed** denotes a value or behavior present in a durable experiment record or source artifact but not necessarily rerun for this paper.
- **Reproduced** denotes a check executed on the current local tree for this paper.
- **Proposed** denotes a design, hypothesis, or future experiment not established by current evidence.

Repository-relative paths refer to `projects/chaos-language-algorithm/repos/chaoslang`. The inspected branch was `agent/hd-embedding-cla`, HEAD `16fe6bc`; `origin/main` remained at `847390c`. Pre-existing untracked `docs/` content was not treated as committed provenance.

2 Motivation

Symbolic dynamics replaces a trajectory with an itinerary through a finite partition. CLA asks whether that itinerary admits reusable sequential and categorical structure. This is narrower than proving chaos and broader than counting repeated words: the output must be an inspectable grammar and parse whose expansion is the original symbolic observation.

The immediate project motivation is detector calibration for OmegaSim. Project decision records pause interpretation of simulated OmegaHive dynamics until a detector can first be assessed on known systems. This ordering is methodologically important. Compression of one finite trajectory does not by itself prove a strange attractor, identify a generating partition, or establish predictive generalization. CLA is presently a grammar-induction instrument, not a validated chaos classifier.

3 Conceptual model

Let $U = (u_1, \ldots, u_T)$ be a trajectory. Optional delay reconstruction gives

$$x_t = (u_t, u_{t-\tau}, \ldots, u_{t-(d-1)\tau}) \in \mathbb{R}^d. \tag{1}$$

A partition $\mathcal{P} = \{C_1, \ldots, C_k\}$ and label map ϕ produce the base string

$$S_0 = s_1 \cdots s_n, \qquad s_t = \phi(x_t). \tag{2}$$

CLA searches for a grammar G and exact parse P minimizing a two-part objective

$$L(G, P) = L_{\text{model}}(G) + L_{\text{data}}(P \mid G), \tag{3}$$

subject to $\text{expand}(G, P) = S_0$.

3.1 Chunks and categories

A chunk is a sequential abbreviation

$$N \to v_1 \cdots v_m. \tag{4}$$

Replacing non-overlapping occurrences captures "and then" structure. A category is a set $M = \{v_1, \ldots, v_q\}$ of substitutable tokens. It captures "one of these" structure and is not a production with a long right-hand side. Exact reconstruction requires a category use to retain its observed member, written $M[v]$. Its expansion is v, not the concatenation or arbitrary choice of members. This distinction is represented directly in the implementation types `Production`, `Category`, and `CategoryOccurrence`.

4 Implemented algorithms and data structures

4.1 State and edit discipline

Observed The domain core at `16fe6bc` uses frozen dataclasses for tokens, productions, categories, corpus, score, edit, proposals, and grammar state. The state stores the immutable original corpus, current parse, production and category maps, score, human-readable history, and edit log. Miners return proposals and do not mutate state. `EditApplier` performs rewrites and asserts equality of expansions before and after each edit.

Chunk application inserts a fresh chunk token at selected non-overlapping starts and adds its production. Rule utility is enforced by repeatedly pruning a production used fewer than twice. Commit `c496b59` fixed a material failure: pruning had to inline references in other production right-hand sides as well as in the top-level parse, otherwise nested rules could dangle. The regression is covered by `tests/test_nested_prune.py`.

4.2 Bounded suffix-trie mining

For current parse length n and maximum chunk width m, the miner inserts every suffix prefix up to depth m into a trie. Each node stores start positions. Repeated paths of lengths $n_{\min}$ through m are converted to greedily selected non-overlapping occurrences. For fixed m, construction stores $O(nm)$ node traversals rather than separately materializing all tuple windows for every width. Proposal ordering is deterministic, prioritizing an occurrence-length proxy and then stable keys. Commits `97cfb4f` and `b344aa3` introduced and then made this structure the implementation behind `NGramPatternMiner`; a compatibility `SuffixTrieMiner` path also exists. Tests check representative proposal equivalence, compound symbols, non-overlap, and a linear node bound. No matched wall-time or peak-memory comparison with the historical brute-force implementation has been recorded.

4.3 Category induction

The conservative exact inducer records immediate left/right contexts for each token and groups tokens with identical sets of context pairs, subject to occurrence and member minima. This proposes the intended frame generalization for strings such as `a x b / a y b / a x b`. The JS path computes smoothed context distributions and greedily clusters sorted tokens by Jensen–Shannon divergence to a centroid under a threshold (default 0.1). Candidate positions are rewritten as $M[v]$. The relatively high fixed category overhead means many plausible clusters are rejected by the scorer; in particular, the recorded 1024 by 20D run accepted none.

4.4 Approximate MDL search

At each iteration the learner enumerates chunk proposals and, when enabled, category proposals. Every proposal is applied to a copied immutable state, scored, and checked for exact expansion. The best strictly decreasing total is accepted; deterministic proposal representation breaks ties. Search stops when no proposal improves the score or the iteration limit is reached. Although the API constructs a seeded random generator, the inspected selection loop does not use randomness.

The current scorer is a stable engineering proxy, not a complete prefix code. Ordinary parse entries cost 1.0 unit. A production costs right-hand-side length plus 0.8 units. A category costs member count plus 14.0 units. Edit-log entries cost 0.1 units. For a category occurrence the data cost includes a member term based on $-\log_2 p(v \mid M)$. Accordingly, historical records that call totals "bits" overstate calibration. This paper calls them *score units*, except for Phase-0 diagnostics whose field names explicitly say bits but still combine incompatible coding assumptions.

4.5 Persistence and fact projection

Commits `c6bb08a`/`e64b1d0` add deterministic JSON state serialization, stable text rendering, and edit-log replay. Tests cover empty and fitted round trips, categories, logs, replay, and deterministic rendering. The backend-neutral `Fact` and `MemoryFactStore` project core state into predicate/argument records and reconstruct core fields. This is the implemented seam for later Atomspace storage. No Hyperon runtime adapter, MeTTa miner, or Hyperon-native search is implemented.

5 Symbolization

5.1 Fixed rectangular M1

M1 partitions each axis into equal-width bins and emits one compound label per vector sample. Bounds may be supplied or inferred from the complete trajectory; out-of-range values are clamped. With b bins and ambient dimension D, the possible compound alphabet grows as b^D. Inferring bounds from the whole record also leaks global range information into any nominal train/test split. M1 remains useful as a deterministic baseline, but the observed growth in unique compound symbols makes it unsuitable as the sole high-dimensional method.

5.2 Adaptive microstates

Observed Commit `962b30f` adds a dependency-free deterministic k-means implementation. Farthest-first initialization begins at a seed-selected point; ties are stable. Alphabet cardinality is chosen directly as k, decoupling it from D. Unit tests verify cardinality, determinism, and exact CLA expansion of microstate symbols. These are functional tests, not evidence that the clusters form a generating partition.

6 High-dimensional grammar-preserving embedding

6.1 Rationale and intended pipeline

The high-dimensional design diagnoses M1 collapse primarily as a partition-cardinality failure: recurrence is removed before grammar induction. The recommended pipeline is

$$\begin{aligned}\text{trajectory} &\to \text{intrinsic-dimension diagnostic} \to \text{TICA/VAMP kinetic map} \\ &\to \text{k-means microstates} \to \text{CLA}.\end{aligned}$$

PCCA+ soft memberships would seed category proposals, while chunks continue to represent temporal motifs. Microstates, rather than near-Markovian PCCA+ macrostates, remain CLA's base alphabet.

6.2 Implemented Phase-0 path

Observed Commit `962b30f` implements a spectral participation-ratio diagnostic and a small pure-Python non-reversible kinetic map. For lag τ, it estimates separately centered and shrinkage-regularized C_{00} and $C_{\tau\tau}$ and unsymmetrized $C_{0\tau}$, then forms

$$K = C_{00}^{-1/2} C_{0\tau} C_{\tau\tau}^{-1/2}. \tag{5}$$

A Jacobi eigensolver obtains singular directions; singular values are clipped to $[0, 1]$ to limit rank-deficient whitening artifacts. Coordinates use left singular functions scaled by these values. The implementation deliberately preserves directionality, but its dense pure-Python eigensolver is cubic in ambient dimension and is labeled a small-control fallback.

Commit `02a6ed2` adds an optional `deeptime` backend for VAMP or reversible TICA, implied-timescale diagnostics, sklearn k-means, and PCCA+ memberships. Imports are lazy at function use, preserving a dependency-light core. The tests guarded by `DEEPTIME_AVAILABLE` verify dimensions up to 200, VAMP/TICA selection, timescales, PCCA+ output, and CLI integration when dependencies are available.

6.3 Implemented diagnostics and their limits

The surrogate harness shuffles symbols while preserving marginal counts, refits CLA, and reports real gain minus mean shuffled gain. Its no-grammar baseline is the empirical iid code length, while grammar cost is the current two-part proxy; this is useful for paired diagnostics but is not a final real-bit MDL calibration. The held-out evaluator is a smoothed n-gram next-symbol model, not a predictive CLA likelihood. Its perplexity therefore measures baseline sequence predictability rather than held-out performance of the learned CLA grammar.

6.4 Not yet demonstrated

Proposed The decisive Phase-1 experiment is a Lorenz-63 trajectory lifted by a deterministic random map into approximately $\mathbb{R}^{256}$ with small noise, followed by intrinsic-dimension estimation, VAMP/TICA to $d = 3 \dots 6$, microstate clustering, CLA, shuffled controls, and held-out assessment. Success requires grammar-over-surrogate recovery unavailable to raw M1. No durable experiment record for this battery, no d/k rate-distortion sweep, and no OmegaSim trace result was found. PCCA+ membership computation exists, but automatic conversion of memberships into CLA category proposals is not wired into the main learner.

7 Experimental systems and protocols

The repository implements deterministic generators for the logistic map, Lorenz-63, Rossler, Mackey–Glass, and Lorenz-96. Lorenz-63, Rossler, and Lorenz-96 use fixed-step RK4; Mackey–Glass uses Euler with a discrete delay buffer; the logistic map is iterated directly. The recorded baseline protocol retained 256 samples after discarding 64, used four M1 bins per axis, and allowed eight greedy edits. These records do not include independent integrator validation, Lyapunov estimates, confidence intervals, or raw trajectory artifacts.

System	Dimension	Main parameters	Integrator
Logistic	1	$r = 4.0$, $x_0 = 0.123$	direct map
Lorenz-63	3	$(10, 28, 8/3)$, $dt = 0.01$	RK4
Rossler	3	$(0.2, 0.2, 5.7)$, $dt = 0.05$	RK4
Mackey–Glass	1	$(0.2, 0.1, 10, 17)$, $dt = 0.1$	Euler/delay
Lorenz-96	5–24	$F = 8.0$, $dt = 0.01$	RK4

Table 1: Recorded dynamical controls. A system name does not independently verify chaos in each finite numerical segment.

8 Results

8.1 Recorded M1 experiments

Observed Table 2 transcribes repository and project experiment records. All report exact symbolic reconstruction. Scores are proxy units. The dimension-8 score is lower than dimension 5, so the small suite does not support a monotonic score claim. The robust trend is increasing observed alphabet cardinality and scores near raw parse length at dimensions 16–24.

The first ten records are under `experiments/*/RUN.md` and were committed in the lineage of `0152027` and `1501346`; individual files omit exact commit hashes and environment

System/run	D	n	Unique	Rules	Score	Exact
Mackey–Glass M1	1	256	4	6	44.4	yes
Logistic M1	1	256	4	8	113.2	yes
Lorenz-63 M1	3	256	21	8	187.2	yes
Rossler M1	3	256	24	8	189.2	yes
Lorenz-96 M1	5	256	56	8	234.2	yes
Lorenz-96 scale	8	256	78	8	229.2	yes
Lorenz-96 scale	12	256	107	6	241.4	yes
Lorenz-96 scale	16	256	121	3	248.7	yes
Lorenz-96 scale	20	256	127	6	251.4	yes
Lorenz-96 scale	24	256	148	5	251.5	yes
Lorenz-96 trie/JS	20	1024	510	8	1021.2	yes

Table 2: Recorded runs. "Rules" counts final productions; no category was accepted in the final row.

captures. The 1024 by 20D project-level run is stronger provenance. Its project experiment directory is `20260709T192728Z-lorenz96-1024-dim20-suffix-trie`. The record identifies commit `b344aa3c1a4b5a5eeef93fc2ea239a7f75c97429`, a dirty CLI-only modification, exact command, Pop!_OS/Python environment, exit status zero, empty stderr, and model fit wall time 2.554301110096276 seconds. It establishes completion and exactness, not a trie speedup or JS benefit.

8.2 Current test reproduction

Reproduced On 2026-07-12, at commit `16fe6bc`, the command

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

reported `Ran 102 tests in 38.181s` and `OK`. The host used Python 3.10.12. Coverage includes core identities, exact expansion, frame-category proposal, MDL rejection, dead-rule pruning, deterministic behavior, M1 controls, JS clustering/integration, persistence/replay, bounded-trie behavior, adaptive symbolization, kinetic-map shapes, surrogate and held-out harnesses, CLI paths, and parametric sweep smoke tests.

The run printed deterministic parametric-sweep diagnostics because tests call sweep functions. Examples include logistic $r = 2.5$ and 3.2 with three rules and held-out perplexity 1.00, $r = 4.0$ with five rules and perplexity 2.37, and Lorenz-96 $F = 4.0$ with five rules and perplexity 13.98. These are test fixtures with varying sample lengths and iteration budgets, not a single preregistered experiment. They should not be interpreted as a monotone complexity law. Commit `16fe6bc` adds the sweep harness and tests, but no durable sweep result artifact or environment record.

8.3 Material failures and caveats

1. **Observed** Nested pruning initially risked dangling nonterminals; commit `c496b59` fixed inlining inside productions and added regression coverage.

2. **Observed** Fixed M1 in high dimension produced 510 unique compound symbols in 1024 positions and score close to raw length. This is evidence of a symbolization failure mode, not evidence that Lorenz-96 or CLA lacks grammar.

3. **Observed** The 1024 by 20D run accepted no category, so it does not validate JS categories as useful on high-dimensional M1 streams.

4. No empirical brute-force/trie timing or peak-RSS comparison exists.

5. Historical claims of "bits/symbol" are not justified by the current scorer. Dividing proxy score by n yields units per symbol, not entropy.

6. The baseline suite lacks periodic/non-chaotic controls paired under one protocol, shuffled controls, repeated seeds, confidence intervals, and held-out CLA likelihood.

7. M1 bounds inferred from the complete trajectory prevent clean out-of-sample symbolization.

8. Optional backend tests depend on installed packages. The aggregate current result is 102 tests passing; verbose unittest output does not provide a concise per-backend executed/skip count in the saved summary, so this paper does not claim that every optional deeptime test ran.

9. The pure-Python kinetic map clips singular values and uses a custom Jacobi solver. Numerical agreement with a trusted implementation is not established by shape and smoke tests.

10. The optional PCCA+ function catches broad exceptions and returns empty memberships. Production use should expose diagnostics rather than silently treating all failures alike.

9 Architecture and future Hyperon integration

The architecture separates a stable domain core from proposal generators, scorers, stores, and optional backends. The implemented fact layer supports projection to ordinary facts and in-memory querying without making Hyperon a core dependency. A staged integration can preserve this boundary:

1. implement `HyperonFactStore` with exact state/fact round-trip parity;

2. express context and repeated-path queries in MeTTa while returning ordinary typed proposals;

3. compare Python and Hyperon proposal sets and scores on fixed fixtures;

4. move search or inference only after semantic parity and reproducibility are demonstrated;

5. retain Python expansion as an independent oracle during migration.

Proposed Hyperon may later supply metagraph pattern matching, category inference, distributed fact storage, or richer proposal search. It should not alter chunk/category semantics or bypass exact reconstruction and MDL acceptance.

10 Research agenda

Priority experiments are ordered to close current evidential gaps.

1. Record the 256-dimensional lifted Lorenz-63 ground-truth experiment, including raw/embedded artifacts, hashes, seeds, exact commit, environment, and matched raw-M1 control.

2. Sweep embedding dimension d, microstate count k, lag, and sample interval; report surrogate excess compression and a genuine CLA predictive code where available.

3. Add periodic, quasiperiodic, stochastic, and shuffled controls with repeated seeds and uncertainty intervals.

4. Replace proxy MDL with a complete consistent code; separate model selection from reporting and calibrate all terms to bits.

5. Benchmark historical window counting against the bounded trie using identical proposals, wall time, allocations, and peak RSS.

6. Wire PCCA+ memberships into explicit category proposals and compare against exact and JS categories under matched overheads.

7. Implement logged per-iteration MDL trajectories and test McBride-style approximate delta pre-ranking. Joint chunk/category moves are a hypothesis for escaping greedy barriers, not a proved improvement.

8. Only after ground-truth calibration, ingest versioned OmegaSim traces and report failures as well as positive grammar findings.

11 Conclusion

The implemented CLA is a coherent exact-reconstructing grammar learner with a clear semantic distinction between repeated sequences and substitutable categories. Its strongest demonstrated property is engineering correctness: accepted rewrites preserve the original symbolic stream, and the current 102-test suite passes. Recorded attractor controls show that direct compound M1 symbols become sparse in moderate dimension, motivating adaptive microstates and dynamics-aware embeddings. Those components now exist as Phase-0 code, including an optional production-oriented backend, but the central high-dimensional scientific claim remains untested: that the embedding preserves recoverable grammar better than raw M1 under surrogate and held-out controls. The appropriate conclusion is therefore measured. CLA has progressed from specification to a tested research prototype and a credible experimental pipeline; it is not yet a validated detector of chaotic grammar or an established basis for claims about OmegaSim dynamics.

Reproducibility and provenance summary

- Current code: local branch `agent/hd-embedding-cla`, commit `16fe6bc`.
- Core milestone commits: `9394ab4`, `75884fd`, `713b8bb`, `c496b59`, `d1fd8f9`, `e64b1d0`, `97cfb4f`, `b344aa3`, `962b30f`, `02a6ed2`, `16fe6bc`.
- Current verification: 102 tests passed in 38.181 seconds on Python 3.10.12.
- Strongest recorded run: project experiment `20260709T192728Z-lorenz96-1024-dim20-suffix-trie`, at `b344aa3` plus a recorded CLI-only dirty change.
- Source specification SHA-256: `468c5f49ec7d484bb58a6bc53e28a13f2bba1512be898791ddaea2e96c8af510`.
- Architecture source SHA-256: `3beacd7855f2fa3912d43d026f175b5518ba08216e19f741540a5de7d20d97c4`.

- High-dimensional design: working extracted text; original attachment hash unavailable.

References

[1] B. Goertzel with GPT-5.5-Pro, *The Chaos Language Algorithm: An Implementation-Oriented Specification*, local project document received 2026-07-03.

[2] B. Goertzel and Collaborators, *A Hyperon-Ready Python Architecture for the Chaos Language Algorithm*, local project document, 2026-07-03.

[3] Chaos Language Algorithm Project, *Grammar-Preserving Dimensional Embedding for Strange-Attractor Language Analysis in High Dimension*, local working text, 2026-07-10.

[4] Chaos Language Algorithm Project, *chaoslang source repository and experiment records*, commits through `16fe6bc`, inspected 2026-07-12.

[5] G. Perez-Hernandez, F. Paul, T. Giorgino, G. De Fabritiis, and F. Noe, "Identification of slow molecular order parameters for Markov model construction," *Journal of Chemical Physics*, 139:015102, 2013.

[6] H. Wu and F. Noe, "Variational approach for learning Markov processes from time series data," *Journal of Nonlinear Science*, 30:23–66, 2020.

[7] J. Rissanen, *Stochastic Complexity in Statistical Inquiry*, World Scientific, 1989.